

GEMV with Collective Communications

Contents

- [layout.csl](#)
- [pe.csl](#)
- [run.py](#)
- [commands.sh](#)

This example shows a CSL program that uses collective communications to perform a generalized matrix-vector (GEMV) multiplication operation of the form:

$$y = Ax + b$$

where:

- **A** is a tensor of shape [M, N] (stored distributed on PE memory).
- **x** is a tensor of shape [N, 1]. It is placed in the memory of the northwesternmost PE before computation begins, and then scattered using collective communications.
- **b** is a tensor of shape [M, 1]. It is placed in the memory of the northwesternmost PE before computation begins, and then scattered using collective communications.
- **y** is the output tensor of shape [M, 1]. At the end of computation, it is located in the memory of the southeasternmost PE.

For simplicity, we choose N as a multiple of the width of the kernel and M as a multiple of the height of the kernel. With the default compile parameters for this example, M = 32, N = 16 and we use a PE rectangle of size 4x4 for the kernel. The parameters specifying these values can be modified at compile time.

Note that this algorithm and the implementation are not optimized for performance. It is intended to serve as a non-trivial introductory example of the collectives library.

All computations are done in FP32 format.

The matrix **A**, of shape [M, N], is distributed across the PE memories as follows:

- The first dimension of **A**, M rows, is distributed across the height of the kernel.
- The second dimension of **A**, N columns, is distributed across the width of the kernel.

Since we know that M is 32 and the height of the kernel is 4, each PE will be assigned $32 \div 4 = 8$ rows of **A**.

Similarly, each PE will get $16 \div 4 = 4$ columns of **A**. This means each PE is assigned an 8×4 chunk of the original matrix **A**.

layout.csl

```
// This does  $y = Ax + b$  on a kernel_cols x kernel_rows rectangle of PEs.

// color/ task ID map
//
// ID var          ID var          ID var          ID var
// 0 c2d_x_color_0  9 EXIT           18             27 reserved (memcpy)
// 1 c2d_x_color_1  10 scatter_x     19             28 reserved (memcpy)
// 2               11 broadcast_x_down 20             29 reserved
// 3               12 compute        21 reserved (memcpy) 30 reserved (memcpy)
// 4 c2d_y_color_0  13 gather_result  22 reserved (memcpy) 31 reserved
// 5 c2d_y_color_1  14 c2d_x_entrypt_0 23 reserved (memcpy) 32
// 6               15 c2d_x_entrypt_1 24             33
// 7               16 c2d_y_entrypt_0 25             34
// 8               17 c2d_y_entrypt_1 26             35

// Kernel rectangle of PEs
param kernel_rows : u16; // Height of kernel
param kernel_cols : u16; // Width of kernel

// Global matrix dimensions
param matrix_rows : u16; // Height of matrix
param matrix_cols : u16; // Width of matrix

comptime {
    // Number of matrix rows must be multiple of kernel rectangle height
    @comptime_assert(matrix_rows % kernel_rows == 0);

    // Number of matrix cols must be multiple of kernel rectangle width
    @comptime_assert(matrix_cols % kernel_cols == 0);
}

// Local matrix dimensions
const Mt: u16 = matrix_rows / kernel_rows; // Number of rows per PE
const Nt: u16 = matrix_cols / kernel_cols; // Number of columns per PE

const memcpy = @import_module( "<memcpy/get_params>", .{
    .width = kernel_cols,
    .height = kernel_rows
});

const c2d = @import_module("<collectives_2d/params>");

layout {
    @set_rectangle(kernel_cols, kernel_rows);

    var Px: u16 = 0;
    while (Px < kernel_cols) : (Px += 1) {
        var Py: u16 = 0;
```

```

const memcpy_params = memcpy.get_params(Px);
while (Py < kernel_rows) : (Py += 1) {
    const c2d_params = c2d.get_params(Px, Py, .{
        .x_colors      = .{ @get_color(0),      @get_color(1) },
        .x_entrypoints = .{ @get_local_task_id(14), @get_local_task_id(15) },
        .y_colors      = .{ @get_color(4),      @get_color(5) },
        .y_entrypoints = .{ @get_local_task_id(16), @get_local_task_id(17) },
    });
    @set_tile_code(Px, Py, "pe.csl", .{
        .memcpy_params = memcpy_params,
        .c2d_params    = c2d_params,
        .Mt             = Mt,
        .Nt             = Nt
    });
}
}

// export symbol names
@export_name("A", [*]f32, true);
@export_name("x", [*]f32, true);
@export_name("b", [*]f32, true);
@export_name("y", [*]f32, true);
@export_name("main", fn()void);
}

```

pe.csl

```

param memcpy_params;
param c2d_params;

// Size of our local tile of `A`
param Mt: u16; // Height of local matrix, or num rows per PE
param Nt: u16; // Width of local matrix, or num cols per PE

// Task IDs
const EXIT:          local_task_id = @get_local_task_id(9);
const scatter_x_task_id: local_task_id = @get_local_task_id(10);
const broadcast_x_down_task_id: local_task_id = @get_local_task_id(11);
const compute_task_id:   local_task_id = @get_local_task_id(12);
const gather_result_task_id: local_task_id = @get_local_task_id(13);

const sys_mod = @import_module("<memcpy/memcpy>", memcpy_params);

// The default values of queue IDs and DSR IDs of collectives_2d are applied implicitly.
// See mpi_x and mpi_y of gemm-collectives_2d/pe.csl for default values of queue and DSR IDs.
const mpi_x = @import_module("<collectives_2d/pe>", .{ .dim_params = c2d_params.x });
const mpi_y = @import_module("<collectives_2d/pe>", .{ .dim_params = c2d_params.y });

// Size of rectangle of PEs on which our kernel runs
const Pw = @get_rectangle().width; // Num cols of PEs in kernel
const Ph = @get_rectangle().height; // Num rows of PEs in kernel

```

```

// Only PE (0,0) will be initialized by run.py with a full copy of `x`
var x_src = @zeros([Nt * Pw]f32);
var ptr_x : [*]f32 = &x_src;

// Only PE (0,0) will be initialized by run.py with a full copy of `b`
var b_src = @zeros([Mt * Ph]f32);
var ptr_b : [*]f32 = &b_src;

// Each PE has its own tile of `A` initialized by run.py
var A_tile = @zeros([Mt*Nt]f32);
const dsd_A_tile = @get_dsd(mem1d_dsd, .{ .tensor_access = |i|{Mt} -> A_tile[i*@as(i16, Nt)]
});
var ptr_A : [*]f32 = &A_tile;

// The tile of `x` which will be scattered across PEs received in `scatter_x()`
// or received in `broadcast_x_down()`
var x_tile = @zeros([Nt]f32);
const dsd_x_tile = @get_dsd(mem1d_dsd, .{ .tensor_access = |i|{Nt} -> x_tile[i]});

// The tile of `b` which will be scattered across PEs received in `scatter_b()`
var b_tile = @zeros([Mt]f32);
const dsd_b_tile = @get_dsd(mem1d_dsd, .{ .tensor_access = |i|{Mt} -> b_tile[i]});

// The product of `A_tile` with `x_tile` (computed by the `compute()` task)
var local_prod = @zeros([Mt]f32);
const dsd_local_prod = @get_dsd(mem1d_dsd, .{ .tensor_access = |i|{Mt} -> local_prod[i]});

// The sum of products across a row of PEs
var row_sum = @zeros([Mt]f32);

// The final result is stored on PE (Pw-1, Ph-1)
var final_result = @zeros([Mt * Ph]f32);
var ptr_y : [*]f32 = &final_result;

// Updated at runtime to store x and y IDs of PE reported by collectives library
var px: u16;
var py: u16;

// Entrypoint into kernel
// PE (0,0) scatters `b` into tiles across the left column of PEs
fn main() void {
    mpi_x.init();
    mpi_y.init();
    px = mpi_x.pe_id;
    py = mpi_y.pe_id;

    if (px == 0) {
        mpi_y.scatter(0, @ptrcast([*]u32, &b_src), @ptrcast([*]u32, &b_tile),
            Mt, scatter_x_task_id);
    } else {
        @activate(scatter_x_task_id);
    }
}

// Scatter `x` into tiles across the top row of PEs

```

```

task scatter_x() void {
    if (py == 0) {
        mpi_x.scatter(0, @ptrcast([*]u32, &x_src), @ptrcast([*]u32, &x_tile),
                      Nt, broadcast_x_down_task_id);
    } else {
        @activate(broadcast_x_down_task_id);
    }
}

// Broadcast tiles of `x` down the columns of PEs
task broadcast_x_down() void {
    mpi_y.broadcast(0, @ptrcast([*]u32, &x_tile), Nt, compute_task_id);
}

// Compute the product of the local `x_tile` with the local `A_tile`,
// then reduce it across rows of PEs
task compute() void {

    for (@range(i16, Nt)) |j| {
        // offset dsd_A_tile to the corresponding column of A_tile
        const dsd_A_offset = @increment_dsd_offset(dsd_A_tile, j, f32);
        @fmacs(dsd_local_prod, dsd_local_prod, dsd_A_offset, x_tile[j]);
    }

    if (px == 0) {
        @fadds(dsd_local_prod, dsd_local_prod, dsd_b_tile);
    }

    mpi_x.reduce_fadds(Pw - 1, @ptrcast([*]f32, &local_prod), @ptrcast([*]f32, &row_sum),
                      Mt, gather_result_task_id);
}

// Gather the product into the bottom right PE
task gather_result() void {
    mpi_y.gather(Ph - 1, @ptrcast([*]u32, &row_sum), @ptrcast([*]u32, &final_result),
                Mt, EXIT);
}

task f_exit() void {
    sys_mod.unblock_cmd_stream();
}

comptime {
    @bind_local_task(scatter_x, scatter_x_task_id);
    @bind_local_task(broadcast_x_down, broadcast_x_down_task_id);
    @bind_local_task(compute, compute_task_id);
    @bind_local_task(gather_result, gather_result_task_id);
    @bind_local_task(f_exit, EXIT);

    @export_symbol(ptr_A, "A");
    @export_symbol(ptr_x, "x");
    @export_symbol(ptr_b, "b");
    @export_symbol(ptr_y, "y");
    @export_symbol(main);
}

```

run.py

```
#!/usr/bin/env cs_python

import argparse
import json
import numpy as np

from cerebras.sdk.runtime.sdkruntimepybind import SdkRuntime      # pylint: disable=no-name-in-module
from cerebras.sdk.runtime.sdkruntimepybind import MemcpyDataType # pylint: disable=no-name-in-module
from cerebras.sdk.runtime.sdkruntimepybind import MemcpyOrder    # pylint: disable=no-name-in-module

parser = argparse.ArgumentParser()
parser.add_argument("--name", help="the test name")
parser.add_argument("--cmaddr", help="IP:port for CS system")
args = parser.parse_args()

# Get params from compile metadata
with open(f"{args.name}/out.json", encoding='utf-8') as json_file:
    compile_data = json.load(json_file)

# Kernel rectangle and matrix dimensions
kernel_rows = int(compile_data['params']['kernel_rows'])
kernel_cols = int(compile_data['params']['kernel_cols'])
matrix_rows = int(compile_data['params']['matrix_rows'])
matrix_cols = int(compile_data['params']['matrix_cols'])

# Create tensors for A, X, B.
# Use a deterministic seed so that CI results are predictable
np.random.seed(seed=7)

A = np.random.rand(matrix_rows, matrix_cols).astype(np.float32)
X = np.random.rand(matrix_cols).astype(np.float32)
B = np.random.rand(matrix_rows).astype(np.float32)

# Compute expected result
y_expected = (A @ X) + B

# Specify path to ELF files, set up runner
runner = SdkRuntime(args.name, cmaddr=args.cmaddr)

memcpy_dtype = MemcpyDataType.MEMCPY_32BIT
memcpy_order = MemcpyOrder.ROW_MAJOR

symbol_A = runner.get_id("A")
symbol_x = runner.get_id("X")
symbol_b = runner.get_id("b")
symbol_y = runner.get_id("y")
```

```

runner.load()
runner.run()

print("Copying data...")
# Compute number of rows and cols of A on each PE
per_pe_rows = matrix_rows // kernel_rows
per_pe_cols = matrix_cols // kernel_cols

# This transformation on A creates a flattened array so that the matrix can
# be mapped onto the PEs using MemcpyOrder.ROW_MAJOR copy ordering.
# The arrays holding A on each PE are 1D arrays that store the submatrices
# in row major order.

# As an example, consider A[4, 4], mapped onto a 2x2 grid of PEs:
#
#   Matrix A on host           2 x 2 PE grid, row major A submatrices
# +---+---+---+---+           +-----+-----+
# | 0 | 1 | 2 | 3 |           | PE (0, 0):   | PE (1, 0):   |
# +---+---+---+---+           | 0, 1, 4, 5 | 2, 3, 6, 7 |
# | 4 | 5 | 6 | 7 |           +-----+-----+
# +---+---+---+---+   ----> +-----+-----+
# | 8 | 9 |10 |11 |           | PE (0, 1):   | PE (1, 1):   |
# +---+---+---+---+           | 8, 9,12,13 |10,11,14,15 |
# |12 |13 |14 |15 |           +-----+-----+
# +---+---+---+---+           +-----+-----+
#
# MemcpyOrder.ROW_MAJOR copy ordering maps an input array to dimensions h x w x l,
# with l varying fastest, where:
# - h is kernel_rows (i.e., height of program rectangle)
# - w is kernel_cols (i.e., width of program rectangle)
# - l is per_pe_rows * per_pe_cols (i.e., num elems copied to each PE)
#
# So our input array for memcpy_h2d must be ordered as follows:
# [ 0, 1, 4, 5, 2, 3, 6, 7, 8, 9, 12, 13, 10, 11, 14, 15 ]

# The transformation here takes the matrix A and:
# 1. splits A into kernel_cols submatrices, along the vertical axis
# 2. stacks them into a 3D array, with kernel_col as 0th dimension
# 3. splits 3D array into kernel_rows subarrays, along the horizontal axis
# 4. stacks them into a 4D array, with kernel_row as 0th dimension
# 5. flattens into a 1D array
data = np.stack(np.split(np.stack(np.split(A, kernel_cols, axis=1)), kernel_rows,
axis=1)).ravel()

# Copy flattened A array onto PEs
runner.memcpy_h2d(symbol_A, data, 0, 0, kernel_cols, kernel_rows, per_pe_rows * per_pe_cols,
streaming=False, data_type=memcpy_dtype, nonblock=False,
order=memcpy_order)

# Place x and b on PE (0,0). They will be scattered with collective comms
runner.memcpy_h2d(symbol_x, X, 0, 0, 1, 1, matrix_cols,
streaming=False, data_type=memcpy_dtype, nonblock=False, order=memcpy_order)
runner.memcpy_h2d(symbol_b, B, 0, 0, 1, 1, matrix_rows,
streaming=False, data_type=memcpy_dtype, nonblock=False, order=memcpy_order)

```

```

print("Launching kernel...")
# Run the kernel
runner.launch("main", nonblock=False)

# Collect the result y from PE (kernel_cols-1,kernel_rows-1) and compare to expected
y = np.zeros(matrix_rows, dtype=np.float32)
runner.memcpy_d2h(y, symbol_y, kernel_cols-1, kernel_rows-1, 1, 1, matrix_rows,
                  streaming=False, data_type=memcpy_dtype, nonblock=False,
                  order=memcpy_order)

runner.stop()
print("Copied back result.")

print("y calculated: ", y)
print("y expected:  ", y_expected)
np.testing.assert_allclose(y, y_expected, atol=0.01, rtol=0)
print("SUCCESS")

```

commands.sh

```

#!/usr/bin/env bash

set -e

cslc --arch=wse3 ./layout.csl --fabric-dims=11,6 --fabric-offsets=4,1 \
--params=kernel_rows:4,kernel_cols:4,matrix_rows:32,matrix_cols:16 \
--memcpy --channels=1 -o out
cs_python run.py --name out

```